



Data Visualisation

**matplotlib #seaborn #python
#datascience**

Why Visualise?

Goal	Example
Find patterns	Pair plots, correlation heatmaps
Communicate insights	Bar charts in presentations

Exploratory → **for you (raw, fast)** | **Explanatory** → **for others (polished, focused)**

[!tip] Always ask: "What is the one thing I want the viewer to understand?"

Matplotlib Basics

```
import matplotlib.pyplot as plt
import numpy as np
```

Figure Setup

```
plt.figure(figsize=(10, 6))  # Width x Height in inches
plt.tight_layout()           # Prevents label overlap
plt.grid(True)               # Adds gridlines
plt.savefig('plot.png', dpi=150, bbox_inches='tight') # Save to file
plt.show()
```

Labels & Limits

```
plt.xlabel("Year")
plt.ylabel("Runs")
plt.title("My Plot")
plt.legend()
plt.xlim(1990, 2010)  # Set x-axis range
plt.ylim(0, 2000)     # Set y-axis range
```

Styles

```
print(plt.style.available)
plt.style.use("ggplot")      # Clean R-like style
plt.style.use("seaborn-v0_8-bright")

with plt.xkcd():             # Comic style (fun!)
    plt.plot(x, y)
plt.show()
```

Line Plot

```
years = [1990, 1992, 1994, 1996, 1998, 2000]
runs  = [500, 700, 1100, 1500, 1800, 1200]

plt.plot(years, runs)                                # Basic
plt.plot(years, runs, color='orange', linestyle='--', linewidth=2, label="Sachin")
plt.plot(years, runs, 'ro--', label="Sachin")        # Format string shorthand
```

Format Strings — '[color][marker][linestyle]'

Symbol	Meaning	Symbol	Meaning
r g b	red / green / blue	o	circle marker
--	dashed	^	triangle marker
-.	dash-dot	:	dotted

```
# Multiple lines
kohli  = [0, 500, 800, 1100, 1300, 1500]
sehwag = [300, 800, 1200, 1500, 1700, 1600]

plt.plot(years, kohli, 'ro--', label="Kohli")
plt.plot(years, sehwag, 'g^:', label="Sehwag")
plt.legend()    # Always use label= on each plot, not plt.legend(["a","b"])
plt.show()
```

Annotations

```
plt.annotate("Peak",
             xy=(1998, 1800),          # Point to annotate
             xytext=(1996, 1900),     # Where to put the text
             arrowprops=dict(arrowstyle="->", color="black"))
```

Bar Chart

```
# Vertical
plt.bar(years, runs, color='steelblue', edgecolor='black')
plt.xticks(rotation=45)    # Rotate x labels if crowded

# Horizontal (good for long category names)
players = ["Sachin", "Sehwag", "Kohli"]
runs_tot = [18000, 8000, 12000]
plt.barh(players, runs_tot, color="skyblue")
```

Grouped Bar Chart

```
sachin = [500, 700, 1100, 1500, 1800]
sehwag = [0, 200, 900, 1400, 1600]
kohli = [0, 0, 500, 800, 1100]
x = np.arange(5)
width = 0.25

plt.bar(x - width, sachin, width=width, label="Sachin")
plt.bar(x, sehwag, width=width, label="Sehwag")
plt.bar(x + width, kohli, width=width, label="Kohli")
plt.xticks(x, years[:5])    # Replace 0,1,2... with real labels
plt.legend()
```

Value Labels on Bars

```
for i, val in enumerate(runs_tot):
    plt.text(i, val + 50, str(val), ha='center')    # ha='center' to centre the text
```

Pie Chart

```
labels = ["Sachin", "Sehwag", "Kohli", "Yuvraj"]
runs   = [18000, 8000, 12000, 9500]
colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99']
explode = [0.1, 0, 0, 0] # Pull out first slice

plt.pie(runs,
        labels=labels,
        colors=colors,
        explode=explode,
        shadow=True,
        startangle=140,      # Rotate starting position
        autopct='%1.1f%%',  # Show percentages
        wedgeprops={'edgecolor': 'black', 'linewidth': 1.5})

plt.title("Career Run Share")
plt.show()
```

[!warning] Avoid pie charts with more than 5–6 slices — too cluttered to read. Use a horizontal bar chart instead.

Histogram

```
ages = [34, 28, 36, 45, 27, 25, 32, 10, 12, 24, 19, 33, 20, 15, 44, 27, 30]

plt.hist(ages, bins=10, edgecolor='black') # 10 equal-width bins
plt.hist(ages, bins=[10,20,30,40,50,60], edgecolor='black') # Custom bin edges

plt.axvline(np.mean(ages), color='red', linestyle='--', linewidth=2, label='Mean')
plt.legend()
plt.show()
```

Parameter	Purpose
<code>bins=10</code>	Number of equal-width bins
<code>bins=[10,20,30]</code>	Custom bin edges
<code>edgecolor='black'</code>	Border around each bar
<code>axvline()</code>	Vertical reference line (mean, threshold)

Scatter Plot

```

study_hours = [1, 2, 3, 4, 5, 6, 7, 8, 9]
exam_scores = [40, 45, 50, 55, 60, 65, 75, 85, 90]

# Basic
plt.scatter(study_hours, exam_scores)

# With size + color
sizes = [score * 2 for score in exam_scores]
colors = ['red' if s < 60 else 'green' for s in exam_scores]
plt.scatter(study_hours, exam_scores, s=sizes, c=colors)

# With colormap
plt.scatter(study_hours, exam_scores, c=exam_scores, cmap='viridis')
plt.colorbar(label='Score') # Shows color scale legend

```

[!tip] Browse all colormaps at matplotlib.org/stable/users/explain/colors/colormaps.html

Stack Plot

Use when you want to show **how parts of a whole change over time**.

```

days      = [1, 2, 3, 4, 5, 6, 7]
studying   = [3, 4, 3, 5, 4, 3, 4]
playing    = [2, 2, 1, 1, 2, 3, 2]
watching   = [2, 1, 2, 2, 1, 1, 1]
sleeping   = [5, 5, 6, 5, 6, 5, 5]

plt.stackplot(days, studying, playing, watching, sleeping,
              labels=['Studying','Playing','Watching TV','Sleeping'],
              colors=['skyblue','lightgreen','gold','lightcoral'],
              alpha=0.8)

plt.legend(loc='upper left')
plt.xlabel('Day'); plt.ylabel('Hours')
plt.title('Weekly Activity Tracker')
plt.grid(True)
plt.show()

```

[!note] Pie → snapshot in time. Stack Plot → change over time.

Subplots

Quick Method — `plt.subplot()`

```
plt.subplot(1, 2, 1)    # (rows, cols, plot_number)
plt.plot(x, y1)
plt.title('Plot 1')

plt.subplot(1, 2, 2)
plt.plot(x, y2)
plt.title('Plot 2')

plt.tight_layout()
plt.show()
```

Better Method — `plt.subplots()`

```
fig, axs = plt.subplots(2, 2, figsize=(10, 8))

axs[0, 0].plot(x, y1); axs[0, 0].set_title('x * 2')
axs[0, 1].plot(x, y2); axs[0, 1].set_title('x2')
axs[1, 0].plot(x, y3); axs[1, 0].set_title('√x')
axs[1, 1].plot(x, y4); axs[1, 1].set_title('10 - x')

fig.suptitle('Overview', fontsize=14)    # Title for whole figure
fig.tight_layout()
fig.savefig('my_plots.png')
plt.show()
```

[!note] `axs` controls individual plots. `fig` controls the whole figure.

Looping Over Subplots

```
fig, axs = plt.subplots(2, 2, figsize=(8, 6))
ys      = [y1, y2, y3, y4]
titles = ['x*2', 'x^2', '√x', '10-x']

for i in range(2):
    for j in range(2):
        idx = i * 2 + j
        axs[i, j].plot(x, ys[idx])
        axs[i, j].set_title(titles[idx])

plt.tight_layout()
plt.show()
```

Seaborn

Seaborn = Matplotlib + beautiful defaults + DataFrame-aware.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Themes

```
sns.set_theme(style="darkgrid")
# Options: darkgrid | whitegrid | dark | white | ticks
```

Built-in Datasets

```
print(sns.get_dataset_names())          # See all available

tips    = sns.load_dataset('tips')      # Restaurant bills + tips
flights = sns.load_dataset('flights')  # Monthly passenger counts
iris     = sns.load_dataset('iris')     # Flower measurements
titanic  = sns.load_dataset('titanic')  # Passenger survival
penguins= sns.load_dataset('penguins') # Penguin species data
```

Line Plot

```
sns.lineplot(x="total_bill", y="tip", data=tips)
sns.lineplot(x="total_bill", y="tip", hue="time", data=tips) # colour by column
```

Scatter Plot

```
sns.scatterplot(x="total_bill", y="tip", data=tips)
sns.scatterplot(x="total_bill", y="tip", hue="time", style="sex", size="size", data=tips)
# hue = colour by column | style = marker shape | size = point size
```

Bar Plot

```
sns.barplot(x="day", y="total_bill", data=tips)          # Shows mean + CI
sns.barplot(x="day", y="total_bill", hue="sex", data=tips) # Grouped
sns.countplot(x="day", data=tips)                       # Counts per category
```

[!note] `barplot` shows the **mean** with a confidence interval error bar automatically.

Box Plot

Shows median, quartiles, and outliers in one view.

```
sns.boxplot(x="day", y="total_bill", data=tips)
sns.boxplot(x="day", y="total_bill", hue="sex", data=tips)
```

Violin Plot

Like a box plot but also shows the **distribution shape** on each side.

```
sns.violinplot(x="day", y="total_bill", data=tips)
sns.violinplot(x="day", y="total_bill", hue="sex", split=True, data=tips)
# split=True draws both groups on one violin
```

Histogram & KDE

```
sns.histplot(tips["total_bill"], bins=20, kde=True)      # kde=True adds smooth curve
sns.kdeplot(tips["total_bill"], fill=True)               # Smooth density curve only
sns.kdeplot(tips["total_bill"], hue=tips["time"])        # Compare groups
```

Regression Plot

Scatter plot with a **fitted regression line** — shows trend instantly.

```
sns.regplot(x="total_bill", y="tip", data=tips)          # Simple regression line
sns.lmplot(x="total_bill", y="tip", hue="sex", data=tips) # Regression per group
```

Heatmap

Best for correlation matrices and pivot tables.

```
# Correlation matrix
corr = tips.corr(numeric_only=True)
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5)

# Pivot table (e.g. flights dataset)
pivot = flights.pivot("month", "year", "passengers")
sns.heatmap(pivot, annot=True, fmt="d", cmap="YlGnBu")
```

Pair Plot

Plots **every numeric column against every other** — great for quick EDA.

```
sns.pairplot(iris)                                # All numeric pairs
sns.pairplot(iris, hue="species")                  # Colour points by category
sns.pairplot(iris, hue="species", diag_kind="kde") # KDE on diagonal
```

[!tip] `pairplot` is one of the fastest ways to understand a new dataset at a glance.

Strip & Swarm Plot

Shows all individual data points (good for small datasets).

```
sns.stripplot(x="day", y="total_bill", data=tips, jitter=True) # Points jittered
sns.swarmplot(x="day", y="total_bill", data=tips)              # No overlaps
```

Color Palettes

```
sns.set_palette("pastel")
sns.set_palette("husl")
sns.set_palette("coolwarm")

print(sns.palettes.SEABORN_PALETTES.keys()) # See all built-in palettes

# Use palette inside a plot
sns.barplot(x="day", y="total_bill", palette="Set2", data=tips)
```

Matplotlib vs Seaborn

Feature	Matplotlib	Seaborn
Default style	Basic	Beautiful
Syntax	Low-level	High-level
DataFrame support	Manual	Built-in
Complex plots	Verbose	Easy
Fine-tuning	Full control	Use Matplotlib on top

[!tip] Start with **Seaborn** for speed and looks. Drop into **Matplotlib** for fine-tuning (titles, limits, annotations, saving).

Practice Exercises

- [] Line plot comparing Kohli, Rohit, and Sehwag over 10 years with labels, legend, colours, and a custom style
- [] Side-by-side bar chart for 3 players over 5 years + horizontal bar chart of total runs
- [] Pie chart for mobile OS market share → recreate as horizontal bar chart
- [] Stack plot of your own weekly schedule
- [] Scatter plot with colormap showing age vs salary coloured by gender
- [] Seaborn pairplot on the `iris` dataset coloured by species
- [] Heatmap of the `flights` dataset correlation matrix
- [] Combine a histogram + KDE on the `tips` `total_bill` column